Практическая работа №3: Проектирование логической и физической структуры данных

Цель работы

Научиться проектировать логическую и физическую структуру данных для сложной предметной области с использованием нескольких типов хранилищ (polyglot persistence). Освоить принципы оптимального размещения данных по различным СУБД с учётом специфики каждого хранилища.

По итогам выполнения работы студент должен уметь:

- Анализировать предметную область и выделять сущности и связи
- Строить ER-диаграмму (логическую модель данных)
- Проектировать физическую схему данных с распределением по нескольким хранилищам
- Обосновывать выбор хранилища для каждого участка схемы данных
- Понимать принцип дублирования данных между хранилищами

Предметная область

Система управления учебным процессом университета, включающая следующие сущности:

Сущность	Описание
Университет	Головная организация
Институт	Структурное подразделение университета
Кафедра	Подразделение института
Специальность	Направление подготовки, может быть закреплено за несколькими кафедрами (M:N)
Курс лекций	Учебная дисциплина, привязанная к специальности

Лекция	Отдельное занятие в рамках курса лекций с аннотацией и типом занятия
Материал лекции	Текстовое содержание лекции (тексты, презентации)
Группа студентов	Учебная группа, обучающаяся по определённой специальности
Студент	Обучающийся, входящий в группу
Расписание	Привязка конкретной лекции конкретного курса к группе на дату и время
Журнал посещений	Отметка о посещении студентом конкретного занятия из расписания

Типы занятий: лекция, практическое занятие, лабораторная работа.

Требования к компьютерной технике (например: «требуется компьютерный класс с ОС Linux», «не требуется», «требуется проектор») в рамках данной работы не моделируются в структуре данных, но должны учитываться при проектировании системы в лабораторных работах.

Теоретическая часть

1. Логическая модель данных (ER-диаграмма)

ER-диаграмма (Entity-Relationship) -- это графическое представление сущностей предметной области и связей между ними. Она описывает структуру данных на логическом уровне, без привязки к конкретной СУБД.

Элементы ER-диаграммы:

- **Сущность (Entity)** -- объект предметной области (прямоугольник)
- **Атрибут (Attribute)** -- свойство сущности (овал или список внутри прямоугольника)
- **Связь (Relationship)** -- отношение между сущностями (ромб или линия)
- **Кардинальность** -- количественная характеристика связи (1:1, 1:N, M:N)

2. Физическая модель данных

Физическая модель описывает, как именно данные будут храниться на уровне конкретных СУБД: таблицы, индексы, типы данных, партиционирование, а также распределение данных по разным хранилищам.

3. Принцип polyglot persistence с дублированием данных

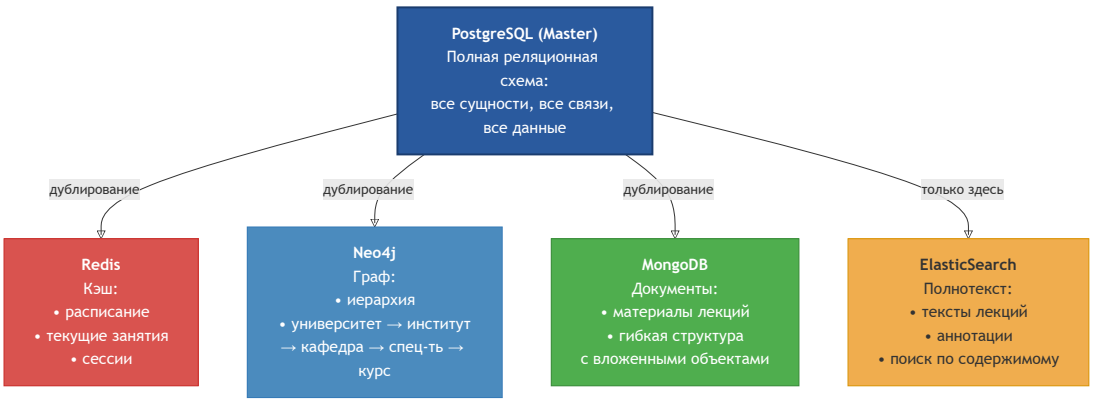
В данной работе применяется следующий подход:

> **PostgreSQL является основным (master) хранилищем, содержащим полную схему данных.** NoSQL-хранилища содержат **дубли отдельных участков** данных, оптимально подходящих для специфических задач доступа.

Это означает:

- Данные из PostgreSQL **могут дублироваться** в Redis, MongoDB, Neo4j, ElasticSearch
- Некоторые данные могут быть **только** в специализированном хранилище (например, полнотекстовые индексы лекций -- только в ElasticSearch)
- PostgreSQL остаётся **источником истины (source of truth)** для всех структурированных данных
- Синхронизация данных между хранилищами реализуется через CDC (Change Data Capture) -- что будет изучено в лабораторных работах 4-6

Схема распределения данных по хранилищам:



4. Обоснование выбора хранилища для каждого участка

Участок данных	PostgreSQL	Redis	Neo4j	MongoDB	ElasticSearch

Университеты, институты, кафедры	master	--	дубль (граф)	--	--
Специальности	master	--	дубль (граф)	--	--
Курсы лекций	master	--	дубль (граф)	--	--
Лекции (метаданные)	master	--	--	--	дубль (поиск)
Аннотации лекций	master	--	--	--	дубль (поиск)
Тексты/ материалы лекций	--	--	--	дубль (документы)	master (полнотекст)
Группы студентов	master	--	--	--	--
Студенты	master	кэш (сессии)	--	--	--
Расписание	master	дубль (кэш)	--	--	--
Журнал посещений	master	дубль (текущее)	--	--	--

Пояснения к выбору:

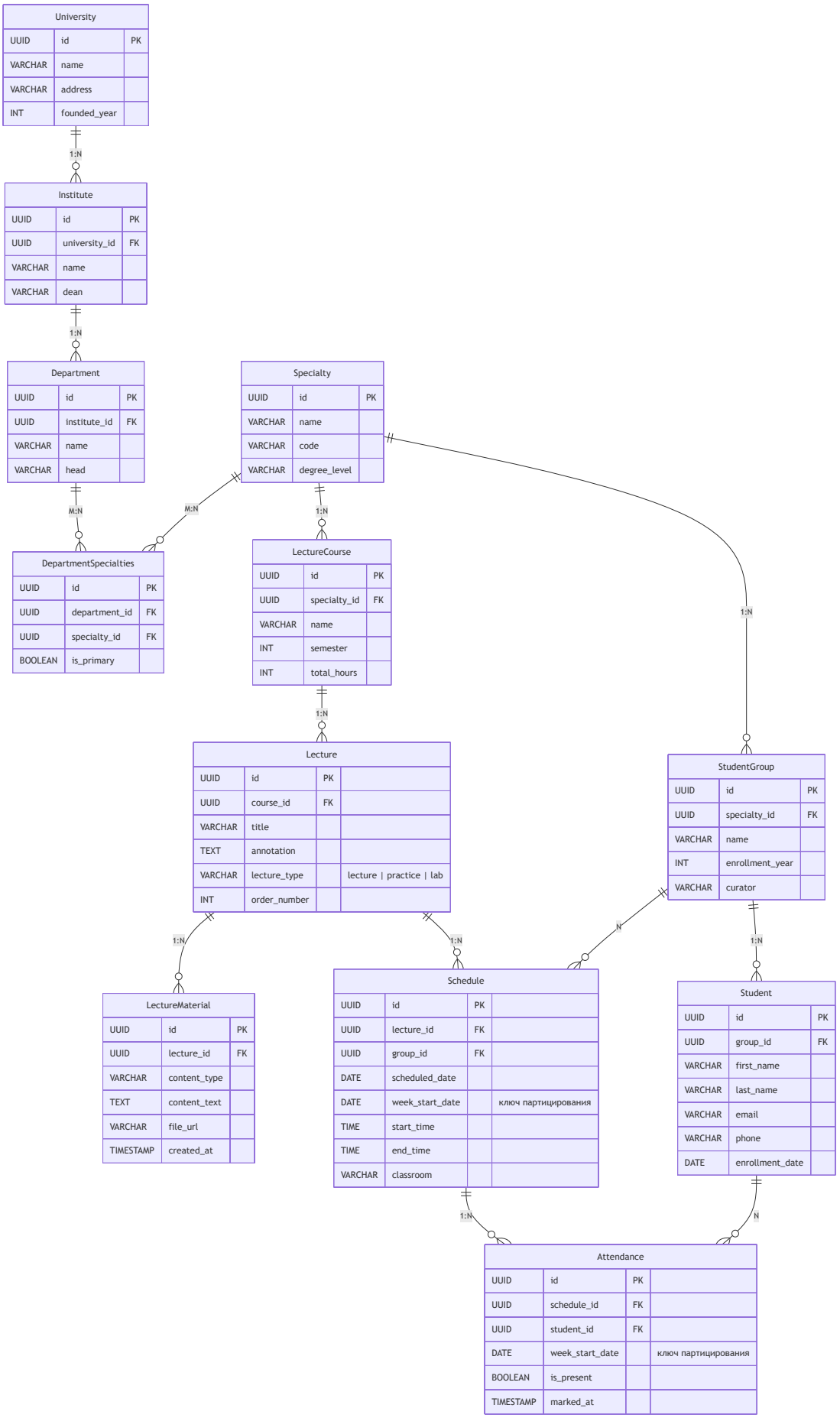
- **PostgreSQL** -- полная схема с referential integrity, ACID-транзакции. Все INSERT/UPDATE/DELETE идут сюда первично.
- **Redis** -- кэширование расписания на текущую неделю (частые чтения), текущие сессии пользователей, кэш посещений для быстрой отметки.
- **Neo4j** -- иерархия «Университет -> Институт -> Кафедра -> Специальность -> Курс лекций» естественно представляется графом. Запросы типа «все курсы в институте X» или «найти кратчайший путь от кафедры до специальности» эффективны на графе.

- **MongoDB** -- материалы лекций имеют произвольную структуру: текст, ссылки на файлы, встроенные медиа, дополнительные метаданные. Документная модель идеальна для хранения таких гетерогенных данных.
- **ElasticSearch** -- полнотекстовый поиск по текстам лекций, аннотациям, названиям курсов. Морфологический анализ, ранжирование по релевантности, подсветка найденного.

Пошаговое выполнение

Шаг 1. Построение ER-диаграммы (логическая модель)

ER-диаграмма описывает все сущности и связи между ними на логическом уровне:


University
UUID id PK
VARCHAR name
VARCHAR address
INT founded_year
1:N
Institute
UUID id PK
UUID university_id FK
VARCHAR name
VARCHAR dean
1:N
Department
UUID id PK
UUID institute_id FK
VARCHAR name
VARCHAR head
Specialty
UUID id PK
VARCHAR name
VARCHAR code
VARCHAR degree_level
M:N
M:N
1:N
1:N
DepartmentSpecialties
UUID id PK
UUID department_id FK
UUID specialty_id FK
BOOLEAN is_primary
LectureCourse
UUID id PK
UUID specialty_id FK
VARCHAR name
INT semester
INT total_hours
1:N
Lecture
UUID id PK
UUID course_id FK
VARCHAR title
TEXT annotation
VARCHAR lecture_type lecture | practice | lab
INT order_number
1:N
1:N
StudentGroup
UUID id PK
UUID specialty_id FK
VARCHAR name
INT enrollment_year
VARCHAR curator
N
1:N
LectureMaterial
UUID id PK
UUID lecture_id FK
VARCHAR content_type
TEXT content_text
VARCHAR file_url
TIMESTAMP created_at
Schedule
UUID id PK
UUID lecture_id FK
UUID group_id FK
DATE scheduled_date
DATE week_start_date ключ партицирования
TIME start_time
TIME end_time
VARCHAR classroom
Student
UUID id PK
UUID group_id FK
VARCHAR first_name
VARCHAR last_name
VARCHAR email
VARCHAR phone
DATE enrollment_date
1:N
N
Attendance
UUID id PK
UUID schedule_id FK
UUID student_id FK
DATE week_start_date ключ партицирования
BOOLEAN is_present
TIMESTAMP marked_at

> **Примечание:** Таблица `Attendance` партицирована (`PARTITION BY RANGE (week_start_date)`). Поле `week_start_date` присутствует как в `Schedule` , так и в `Attendance` для обеспечения партицирования.

Описание связей:

Связь	Кардинальность	Описание
University -> Institute	1:N	В университете несколько институтов
Institute -> Department	1:N	В институте несколько кафедр
Department <-> Specialty	M:N (через department_specialties)	Одна специальность может быть на нескольких кафедрах, одна кафедра может вести несколько специальностей
Specialty -> LectureCourse	1:N	По специальности несколько курсов лекций
Specialty -> StudentGroup	1:N	По специальности несколько групп
LectureCourse -> Lecture	1:N	В курсе несколько лекций (занятий)
StudentGroup -> Student	1:N	В группе несколько студентов
Lecture -> LectureMaterial	1:N	К лекции может быть несколько материалов
Lecture + StudentGroup -> Schedule	M:N (через Schedule)	Расписание связывает лекцию и группу на конкретную дату/время
Schedule + Student -> Attendance	M:N (через Attendance)	Журнал посещений связывает занятие из расписания со студентом

Типы занятий (lecture_type):

Значение	Описание
`lecture`	Лекция -- теоретическое занятие
`practice`	Практическое занятие
`lab`	Лабораторная работа

Шаг 2. Физическая модель -- PostgreSQL (полная схема)

PostgreSQL содержит **полную реляционную модель** -- все сущности и все связи. Это основное хранилище и источник истины.

2.1. Создание базы данных и расширений

```
-- Создание базы данных
CREATE DATABASE university_system;

-- Подключение к базе данных
\c university_system

-- Включение расширений
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";   -- для генерации UUID
CREATE EXTENSION IF NOT EXISTS "pg_trgm";     -- для триграммного поиска
```

Пояснение:

- `uuid-ossp` -- позволяет генерировать UUID в качестве первичных ключей, что удобно при распределённой системе (нет конфликтов ID между хранилищами)
- `pg_trgm` -- позволяет выполнять нечёткий поиск по текстовым полям (полезно для поиска по ФИО)

2.2. Таблицы организационной структуры

```
-- ============================================================
-- Университет
-- ============================================================
CREATE TABLE universities (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```
    name VARCHAR(500) NOT NULL,              -- «МИРЭА — Российский техн
    short_name VARCHAR(100),                 -- «МИРЭА», «РТУ МИРЭА»
    address TEXT,                            -- юридический адрес
    website VARCHAR(255),                    -- https://www.mirea.ru
    founded_year INT,                        -- год основания
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- Пояснение: университет — корневая сущность. В системе может быть
-- несколько университетов (например, для межвузовских программ).

-- ============================================================
-- Институт
-- ============================================================
CREATE TABLE institutes (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    university_id UUID NOT NULL REFERENCES universities(id) ON DELETE CA
    name VARCHAR(500) NOT NULL,              -- «Институт информационных
    short_name VARCHAR(100),                 -- «ИИТ»
    dean VARCHAR(300),                       -- ФИО руководителя (директ
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- Пояснение: институт входит в состав университета.
-- ON DELETE CASCADE — при удалении университета удаляются все его инсти

CREATE INDEX idx_institutes_university ON institutes(university_id);

-- ============================================================
-- Кафедра
-- ============================================================
CREATE TABLE departments (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    institute_id UUID NOT NULL REFERENCES institutes(id) ON DELETE CASCA
    name VARCHAR(500) NOT NULL,              -- «Кафедра программной инж
    short_name VARCHAR(100),                 -- «КПИ»
    head VARCHAR(300),                       -- ФИО заведующего кафедрой
    room VARCHAR(50),                        -- номер кабинета кафедры
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_departments_institute ON departments(institute_id);

-- ============================================================
-- Специальность
-- ============================================================
CREATE TABLE specialties (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```
    name VARCHAR(500) NOT NULL,              -- «Программная инженерия»
    code VARCHAR(20) NOT NULL,               -- «09.03.04»
    degree_level VARCHAR(20) NOT NULL        -- 'bachelor', 'master', 's
        CHECK (degree_level IN ('bachelor', 'master', 'specialist')),
    duration_years INT NOT NULL DEFAULT 4,   -- срок обучения в годах
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

CREATE UNIQUE INDEX idx_specialties_code ON specialties(code);

-- ============================================================
-- Связь кафедра <-> специальность (M:N)
-- ============================================================
-- Одна специальность может быть закреплена за несколькими кафедрами
-- (например, «Программная инженерия» ведётся на кафедре КПИ и на кафедр
-- Одна кафедра может вести несколько специальностей.
CREATE TABLE department_specialties (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    department_id UUID NOT NULL REFERENCES departments(id) ON DELETE CAS
    specialty_id UUID NOT NULL REFERENCES specialties(id) ON DELETE CASC
    is_primary BOOLEAN DEFAULT FALSE,        -- является ли кафедра выпу
    created_at TIMESTAMP DEFAULT NOW(),
    UNIQUE (department_id, specialty_id)      -- одна пара кафедра-специ
);

CREATE INDEX idx_dept_spec_department ON department_specialties(departme
CREATE INDEX idx_dept_spec_specialty ON department_specialties(specialty
```

Пояснение к структуре:

- Каждая таблица имеет `UUID` в качестве первичного ключа -- это важно для синхронизации между хранилищами (один и тот же ID используется в PostgreSQL, Neo4j, Redis, MongoDB)
- `created_at` / `updated_at` -- метки времени для отслеживания изменений, необходимы для CDC
- Каскадное удаление (`ON DELETE CASCADE`) обеспечивает целостность иерархии

2.3. Таблицы учебного процесса

```
-- ============================================================
-- Курс лекций (учебная дисциплина)
-- ============================================================
CREATE TABLE lecture_courses (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    specialty_id UUID NOT NULL REFERENCES specialties(id) ON DELETE CASC
```

```
    name VARCHAR(500) NOT NULL,              -- «Проектирование архитект
    description TEXT,                        -- описание курса
    semester INT NOT NULL                    -- семестр (1-12)
        CHECK (semester BETWEEN 1 AND 12),
    total_hours INT NOT NULL,                -- общее количество часов
    lecture_hours INT DEFAULT 0,             -- часы лекций
    practice_hours INT DEFAULT 0,            -- часы практик
    lab_hours INT DEFAULT 0,                 -- часы лабораторных
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_courses_specialty ON lecture_courses(specialty_id);

-- ============================================================
-- Лекция (отдельное занятие)
-- ============================================================
CREATE TABLE lectures (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    course_id UUID NOT NULL REFERENCES lecture_courses(id) ON DELETE CAS
    title VARCHAR(500) NOT NULL,             -- «Паттерны проектирования
    annotation TEXT,                         -- аннотация к занятию
    lecture_type VARCHAR(20) NOT NULL        -- тип занятия
        CHECK (lecture_type IN ('lecture', 'practice', 'lab')),
    order_number INT NOT NULL,               -- порядковый номер в курсе
    duration_minutes INT DEFAULT 90,         -- длительность в минутах
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_lectures_course ON lectures(course_id);
CREATE INDEX idx_lectures_type ON lectures(lecture_type);

-- ============================================================
-- Материалы лекций (тексты, файлы)
-- ============================================================
CREATE TABLE lecture_materials (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    lecture_id UUID NOT NULL REFERENCES lectures(id) ON DELETE CASCADE,
    content_type VARCHAR(50) NOT NULL        -- тип материала
        CHECK (content_type IN ('text', 'presentation', 'pdf', 'video_li
    title VARCHAR(500),                      -- заголовок материала
    content_text TEXT,                       -- текст материала (для typ
    file_url VARCHAR(1000),                  -- URL файла (для type='pre
    metadata JSONB DEFAULT '{}',             -- дополнительные метаданны
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);
```

```sql
CREATE INDEX idx_materials_lecture ON lecture_materials(lecture_id);
CREATE INDEX idx_materials_type ON lecture_materials(content_type);
```

2.4. Таблицы студентов и групп

```sql
-- ============================================================
-- Группа студентов
-- ============================================================
CREATE TABLE student_groups (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    specialty_id UUID NOT NULL REFERENCES specialties(id) ON DELETE CASC
    name VARCHAR(50) NOT NULL UNIQUE,        -- «ИКБО-01-23», «БСБО-16-2
    enrollment_year INT NOT NULL,            -- год поступления
    curator VARCHAR(300),                    -- ФИО куратора
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_groups_specialty ON student_groups(specialty_id);

-- ============================================================
-- Студент
-- ============================================================
CREATE TABLE students (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    group_id UUID NOT NULL REFERENCES student_groups(id) ON DELETE CASCA
    first_name VARCHAR(100) NOT NULL,        -- имя
    last_name VARCHAR(100) NOT NULL,         -- фамилия
    patronymic VARCHAR(100),                 -- отчество
    email VARCHAR(255) UNIQUE,               -- email
    phone VARCHAR(20),                       -- телефон
    student_card_number VARCHAR(20) UNIQUE,  -- номер студенческого биле
    enrollment_date DATE NOT NULL,           -- дата зачисления
    status VARCHAR(20) DEFAULT 'active'      -- статус
        CHECK (status IN ('active', 'academic_leave', 'expelled', 'gradu
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_students_group ON students(group_id);
CREATE INDEX idx_students_name ON students(last_name, first_name);
CREATE INDEX idx_students_status ON students(status);
```

2.5. Расписание и журнал посещений

```sql
-- ============================================================
-- Расписание занятий
-- ============================================================
-- Поле week_start_date — ключ партицирования, обозначает понедельник
-- той недели, к которой относится занятие. Используется как ключ
-- партицирования в таблице attendance (журнал посещений).
CREATE TABLE schedule (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    lecture_id UUID NOT NULL REFERENCES lectures(id) ON DELETE CASCADE,
    group_id UUID NOT NULL REFERENCES student_groups(id) ON DELETE CASCA
    scheduled_date DATE NOT NULL,            -- дата проведения
    week_start_date DATE NOT NULL,           -- понедельник недели (ключ
    start_time TIME NOT NULL,                -- время начала
    end_time TIME NOT NULL,                  -- время окончания
    classroom VARCHAR(50),                   -- аудитория
    teacher_name VARCHAR(300),               -- ФИО преподавателя
    status VARCHAR(20) DEFAULT 'scheduled'   -- статус занятия
        CHECK (status IN ('scheduled', 'completed', 'cancelled')),
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- Пояснение: каждая строка — это конкретное занятие: группа ИКБО-01-23,
-- лекция №3 курса "Проектирование архитектуры ПО", 15 марта 2026, 10:00
--
-- week_start_date вычисляется при вставке:
--   date_trunc('week', scheduled_date)::DATE
-- Например, для scheduled_date = '2026-03-18' (среда):
--   week_start_date = '2026-03-16' (понедельник этой недели).
-- Это поле денормализовано намеренно — оно служит ключом партицирования
-- для таблицы attendance, что позволяет эффективно работать с журналом
-- посещений за конкретную неделю.

CREATE INDEX idx_schedule_date ON schedule(scheduled_date);
CREATE INDEX idx_schedule_week ON schedule(week_start_date);
CREATE INDEX idx_schedule_group ON schedule(group_id);
CREATE INDEX idx_schedule_lecture ON schedule(lecture_id);
CREATE INDEX idx_schedule_group_date ON schedule(group_id, scheduled_dat
-- Составной индекс для частого запроса «расписание группы на дату»

-- ============================================================
-- Журнал посещений (партицирован по неделям)
-- ============================================================
-- Таблица attendance партицирована по полю week_start_date (RANGE).
-- Каждая партиция содержит записи за одну учебную неделю.
--
-- Зачем партицирование:
-- 1. Журнал посещений — самая быстрорастущая таблица (N студентов × M з
--    За семестр при 500 студентах и 20 занятиях в неделю = ~160 000 стр
--    За год — сотни тысяч. Партицирование ускоряет запросы за конкретну
```

```sql
-- 2. Типичный запрос — «посещаемость за текущую неделю» — обращается то
-- 3. Старые партиции можно архивировать или переносить на медленное хра
-- 4. VACUUM и индексирование работают на уровне партиции, не блокируя в

CREATE TABLE attendance (
    id UUID NOT NULL DEFAULT uuid_generate_v4(),
    schedule_id UUID NOT NULL,               -- ссылка на schedule (без
    student_id UUID NOT NULL,                -- ссылка на students
    week_start_date DATE NOT NULL,           -- ключ партицирования (пон
    is_present BOOLEAN NOT NULL DEFAULT FALSE,  -- присутствовал ли студ
    marked_at TIMESTAMP DEFAULT NOW(),          -- когда была поставлена
    marked_by VARCHAR(300),                     -- кем отмечено (препода
    note VARCHAR(500),                          -- примечание (причина о
    created_at TIMESTAMP DEFAULT NOW(),
    PRIMARY KEY (id, week_start_date)        -- PK включает ключ партици
) PARTITION BY RANGE (week_start_date);

-- Пояснение к ограничениям:
-- В PostgreSQL партицированная таблица не поддерживает FOREIGN KEY на д
-- таблицы (только FROM других таблиц). Поэтому schedule_id и student_id
-- не имеют явного REFERENCES, но целостность обеспечивается на уровне
-- приложения. PRIMARY KEY обязан включать ключ партицирования.

-- Уникальное ограничение: один студент — одна отметка на одно занятие (
CREATE UNIQUE INDEX uq_attendance_schedule_student
    ON attendance(schedule_id, student_id, week_start_date);

CREATE INDEX idx_attendance_schedule ON attendance(schedule_id);
CREATE INDEX idx_attendance_student ON attendance(student_id);
CREATE INDEX idx_attendance_present ON attendance(is_present);
CREATE INDEX idx_attendance_week ON attendance(week_start_date);

-- ============================================================
-- Создание партиций на весенний семестр 2026
-- ============================================================
-- Каждая партиция покрывает одну неделю: [понедельник, следующий понеде

CREATE TABLE attendance_w2026_02_09 PARTITION OF attendance
    FOR VALUES FROM ('2026-02-09') TO ('2026-02-16');  -- неделя 09-15 ф

CREATE TABLE attendance_w2026_02_16 PARTITION OF attendance
    FOR VALUES FROM ('2026-02-16') TO ('2026-02-23');  -- неделя 16-22 ф

CREATE TABLE attendance_w2026_02_23 PARTITION OF attendance
    FOR VALUES FROM ('2026-02-23') TO ('2026-03-02');  -- неделя 23 фев

CREATE TABLE attendance_w2026_03_02 PARTITION OF attendance
    FOR VALUES FROM ('2026-03-02') TO ('2026-03-09');  -- неделя 02-08 м

CREATE TABLE attendance_w2026_03_09 PARTITION OF attendance
    FOR VALUES FROM ('2026-03-09') TO ('2026-03-16');  -- неделя 09-15 м
```

```sql
CREATE TABLE attendance_w2026_03_16 PARTITION OF attendance
    FOR VALUES FROM ('2026-03-16') TO ('2026-03-23');  -- неделя 16-22 м

-- ... и так далее до конца семестра (июнь).
-- В production партиции создаются автоматически скриптом или расширение

-- ============================================================
-- Автоматическое создание партиций (опциональный скрипт)
-- ============================================================
-- Функция для создания партиций на N недель вперёд:
CREATE OR REPLACE FUNCTION create_weekly_partitions(
    start_date DATE,
    num_weeks INT
) RETURNS VOID AS $$
DECLARE
    week_start DATE := date_trunc('week', start_date)::DATE;
    week_end DATE;
    partition_name TEXT;
BEGIN
    FOR i IN 0..num_weeks-1 LOOP
        week_end := week_start + INTERVAL '7 days';
        partition_name := 'attendance_w' || to_char(week_start, 'YYYY_MM

        -- Проверяем, существует ли уже партиция
        IF NOT EXISTS (
            SELECT 1 FROM pg_class WHERE relname = partition_name
        ) THEN
            EXECUTE format(
                'CREATE TABLE %I PARTITION OF attendance FOR VALUES FROM
                partition_name, week_start, week_end
            );
            RAISE NOTICE 'Created partition: %', partition_name;
        END IF;

        week_start := week_end;
    END LOOP;
END;
$$ LANGUAGE plpgsql;

-- Создать партиции на весь семестр (20 недель с 9 февраля 2026):
-- SELECT create_weekly_partitions('2026-02-09', 20);
```

2.6. Представления (Views) для удобных запросов

```sql
-- Полное расписание с развёрнутой информацией
CREATE VIEW v_full_schedule AS
SELECT
```

```
    s.id AS schedule_id,
    s.scheduled_date,
    s.start_time,
    s.end_time,
    s.classroom,
    s.teacher_name,
    s.status AS schedule_status,
    l.title AS lecture_title,
    l.lecture_type,
    lc.name AS course_name,
    lc.semester,
    sg.name AS group_name,
    sp.name AS specialty_name,
    sp.code AS specialty_code,
    d.name AS department_name,
    i.name AS institute_name,
    u.name AS university_name
FROM schedule s
JOIN lectures l ON s.lecture_id = l.id
JOIN lecture_courses lc ON l.course_id = lc.id
JOIN specialties sp ON lc.specialty_id = sp.id
JOIN department_specialties ds ON sp.id = ds.specialty_id AND ds.is_prim
JOIN departments d ON ds.department_id = d.id
JOIN institutes i ON d.institute_id = i.id
JOIN universities u ON i.university_id = u.id
JOIN student_groups sg ON s.group_id = sg.id;

-- Статистика посещаемости по студенту
CREATE VIEW v_student_attendance_stats AS
SELECT
    st.id AS student_id,
    st.last_name || ' ' || st.first_name AS student_name,
    sg.name AS group_name,
    COUNT(a.id) AS total_classes,
    COUNT(CASE WHEN a.is_present THEN 1 END) AS attended,
    COUNT(CASE WHEN NOT a.is_present THEN 1 END) AS missed,
    ROUND(
        COUNT(CASE WHEN a.is_present THEN 1 END)::NUMERIC /
        NULLIF(COUNT(a.id), 0) * 100, 1
    ) AS attendance_percentage
FROM students st
JOIN student_groups sg ON st.group_id = sg.id
LEFT JOIN attendance a ON a.student_id = st.id
GROUP BY st.id, st.last_name, st.first_name, sg.name;
```

Примеры запросов к PostgreSQL:

```sql
-- 1. Расписание группы ИКБО-01-23 на конкретную дату
SELECT * FROM v_full_schedule
WHERE group_name = 'ИКБО-01-23' AND scheduled_date = '2026-03-15';

-- 1a. Посещаемость группы за конкретную неделю (эффективно — обращается
-- PostgreSQL автоматически выполняет partition pruning: при фильтре по
-- запрос сканирует только одну партицию attendance_w2026_03_09, а не вс
SELECT
    st.last_name || ' ' || st.first_name AS student,
    s.scheduled_date,
    l.title AS lecture,
    a.is_present
FROM attendance a
JOIN schedule s ON a.schedule_id = s.id
JOIN lectures l ON s.lecture_id = l.id
JOIN students st ON a.student_id = st.id
JOIN student_groups sg ON st.group_id = sg.id
WHERE a.week_start_date = '2026-03-09'   -- ← partition pruning: только
  AND sg.name = 'ИКБО-01-23'
ORDER BY s.scheduled_date, s.start_time, st.last_name;

-- 2. Все лабораторные работы определённого курса
SELECT l.title, lc.name AS course, l.order_number
FROM lectures l
JOIN lecture_courses lc ON l.course_id = lc.id
WHERE l.lecture_type = 'lab'
ORDER BY l.order_number;

-- 3. Посещаемость группы за семестр
SELECT student_name, attendance_percentage
FROM v_student_attendance_stats
WHERE group_name = 'ИКБО-01-23'
ORDER BY attendance_percentage DESC;

-- 4. Иерархия: все специальности конкретного института
SELECT sp.code, sp.name AS specialty, d.name AS department, ds.is_primar
FROM specialties sp
JOIN department_specialties ds ON sp.id = ds.specialty_id
JOIN departments d ON ds.department_id = d.id
JOIN institutes i ON d.institute_id = i.id
WHERE i.short_name = 'ИИТ';
```

Шаг 3. Физическая модель -- Redis (кэширование)

Redis используется для кэширования часто запрашиваемых данных с коротким временем жизни. Данные в Redis **дублируют** данные из PostgreSQL.

3.1. Что кэшируем и почему

Ключ	Тип данных Redis	Источник (PostgreSQL)	TTL	Обоснование
`schedule:{group_id}:{date}`	Hash	таблица `schedule` + JOIN	1 час	Расписание группы на день запрашивается очень часто (каждый студент группы)
`attendance:{schedule_id}`	Hash	таблица `attendance`	30 мин	Текущая отметка посещаемости для открытого занятия
`session:{user_id}`	String (JSON)	--	24 часа	Сессия пользователя (аутентификация)
`student:{student_id}`	Hash	таблица `students`	2 часа	Данные студента для быстрого доступа
`today_schedule:{group_id}`	List	таблица `schedule` + JOIN	до конца дня	Список занятий группы на сегодня

3.2. Примеры команд Redis

```
-- Кэширование расписания группы на дату
-- Ключ: schedule:{group_id}:{date}
-- Значение: хэш с полями = schedule_id, значение = JSON строка

HSET schedule:550e8400-e29b-41d4-a716-446655440001:2026-03-15 \
    "slot_1" '{"start_time":"08:30","end_time":"10:00","lecture":"Паттер
    "slot_2" '{"start_time":"10:15","end_time":"11:45","lecture":"Базы д

-- Установить TTL (1 час)
EXPIRE schedule:550e8400-e29b-41d4-a716-446655440001:2026-03-15 3600

-- Получить расписание
```

```
HGETALL schedule:550e8400-e29b-41d4-a716-446655440001:2026-03-15

-- Получить конкретный слот
HGET schedule:550e8400-e29b-41d4-a716-446655440001:2026-03-15 "slot_1"

-- ============================================================
-- Кэш отметок посещаемости для текущего занятия
-- ============================================================

-- Когда преподаватель открывает журнал, создаётся кэш:
HSET attendance:schedule_abc123 \
    "student_001" "present" \
    "student_002" "absent" \
    "student_003" "present"

EXPIRE attendance:schedule_abc123 1800  -- 30 минут

-- Отметить студента
HSET attendance:schedule_abc123 "student_004" "present"

-- Получить все отметки
HGETALL attendance:schedule_abc123

-- ============================================================
-- Сессия пользователя
-- ============================================================
SET session:user_12345 '{"user_id":"12345","role":"teacher","name":"Иван
```

Пояснение: Redis не является источником истины. Данные сначала записываются в PostgreSQL, а затем кэшируются в Redis. При истечении TTL данные запрашиваются из PostgreSQL повторно (паттерн Cache-Aside).

Шаг 4. Физическая модель -- Neo4j (граф иерархии)

Neo4j хранит **дубль иерархической структуры** из PostgreSQL: университет -> институт -> кафедра -> специальность -> курс лекций. Графовая модель позволяет эффективно выполнять запросы на обход иерархии.

4.1. Модель графа

Узлы (Nodes):

Метка (Label)	Свойства	Источник в PostgreSQL
`:University`	id, name, short_name	`universities`

`:Institute`	id, name, short_name, dean	`institutes`
`:Department`	id, name, short_name, head	`departments`
`:Specialty`	id, name, code, degree_level	`specialties`
`:LectureCourse`	id, name, semester, total_hours	`lecture_courses`

Связи (Relationships):

Связь	Описание
`(:University)-[:HAS_INSTITUTE]->(:Institute)`	Университет содержит институт
`(:Institute)-[:HAS_DEPARTMENT]->(:Department)`	Институт содержит кафедру
`(:Department)-[:HAS_SPECIALTY {is_primary}]->(:Specialty)`	Кафедра ведёт специальность (M:N, свойство is_primary)
`(:Specialty)-[:HAS_COURSE]->(:LectureCourse)`	Специальность содержит курс лекций

4.2. Создание графа (Cypher)

```
// Создание университета
CREATE (u:University {
    id: '550e8400-e29b-41d4-a716-446655440000',
    name: 'МИРЭА — Российский технологический университет',
    short_name: 'РТУ МИРЭА'
});

// Создание института и связи с университетом
CREATE (i:Institute {
    id: '550e8400-e29b-41d4-a716-446655440010',
    name: 'Институт информационных технологий',
    short_name: 'ИИТ',
    dean: 'Сидоров С.С.'
});

MATCH (u:University {short_name: 'РТУ МИРЭА'})
MATCH (i:Institute {short_name: 'ИИТ'})
CREATE (u)-[:HAS_INSTITUTE]->(i);
```

```
// Создание кафедры
CREATE (d:Department {
    id: '550e8400-e29b-41d4-a716-446655440020',
    name: 'Кафедра программной инженерии',
    short_name: 'КПИ',
    head: 'Иванов И.И.'
});

MATCH (i:Institute {short_name: 'ИИТ'})
MATCH (d:Department {short_name: 'КПИ'})
CREATE (i)-[:HAS_DEPARTMENT]->(d);

// Создание специальности
CREATE (s:Specialty {
    id: '550e8400-e29b-41d4-a716-446655440030',
    name: 'Программная инженерия',
    code: '09.03.04',
    degree_level: 'bachelor'
});

MATCH (d:Department {short_name: 'КПИ'})
MATCH (s:Specialty {code: '09.03.04'})
CREATE (d)-[:HAS_SPECIALTY {is_primary: true}]->(s);

// Пример: другая кафедра тоже ведёт эту специальность (не выпускающая)
// MATCH (d2:Department {short_name: 'КИВ'})
// MATCH (s:Specialty {code: '09.03.04'})
// CREATE (d2)-[:HAS_SPECIALTY {is_primary: false}]->(s);

// Создание курса лекций
CREATE (c:LectureCourse {
    id: '550e8400-e29b-41d4-a716-446655440040',
    name: 'Проектирование архитектуры ПО',
    semester: 6,
    total_hours: 144
});

MATCH (s:Specialty {code: '09.03.04'})
MATCH (c:LectureCourse {name: 'Проектирование архитектуры ПО'})
CREATE (s)-[:HAS_COURSE]->(c);
```

4.3. Запросы к графу

```
// 1. Полная иерархия от университета до курсов лекций
MATCH path = (u:University)-[:HAS_INSTITUTE]->(i:Institute)
    -[:HAS_DEPARTMENT]->(d:Department)
    -[:HAS_SPECIALTY]->(s:Specialty)
    -[:HAS_COURSE]->(c:LectureCourse)
```

```
WHERE u.short_name = 'РТУ МИРЭА'
RETURN u.name, i.name, d.name, s.name, s.code, c.name, c.semester;

// 2. Все курсы лекций конкретного института
MATCH (i:Institute {short_name: 'ИИТ'})-[:HAS_DEPARTMENT]->()-[:HAS_SPEC
RETURN c.name, c.semester
ORDER BY c.semester;

// 3. Найти, к какому институту относится конкретный курс
MATCH (u:University)-[:HAS_INSTITUTE]->(i:Institute)-[:HAS_DEPARTMENT]->
    -[:HAS_SPECIALTY]->()-[:HAS_COURSE]->(c:LectureCourse {name: 'Проект
RETURN u.name AS university, i.name AS institute;

// 4. Все специальности и количество курсов на каждой
MATCH (s:Specialty)-[:HAS_COURSE]->(c:LectureCourse)
RETURN s.name, s.code, COUNT(c) AS courses_count
ORDER BY courses_count DESC;

// 5. Кратчайший путь от университета до конкретного курса (полезно для
MATCH path = shortestPath(
    (u:University {short_name: 'РТУ МИРЭА'})-[*]->(c:LectureCourse {name
)
RETURN [node IN nodes(path) | COALESCE(node.short_name, node.name)] AS b
// Результат: ['РТУ МИРЭА', 'ИИТ', 'КПИ', 'Программная инженерия', 'Прое
```

Почему Neo4j, а не SQL для иерархии?

В PostgreSQL для запроса «все курсы института» нужен JOIN через 4 таблицы. В Neo4j это один обход графа. При глубокой иерархии (например, добавление подкафедр или модулей) графовый запрос масштабируется линейно, а SQL -- экспоненциально.

Шаг 5. Физическая модель -- MongoDB (материалы лекций)

MongoDB хранит **дубль материалов лекций** в виде документов с произвольной структурой. Это удобно, потому что материалы разных лекций имеют разный состав: текст, ссылки на файлы, код, встроенные медиа.

5.1. Структура коллекции

```
// Коллекция: lecture_materials
// Каждый документ — материалы одной лекции (агрегат)

db.lecture_materials.insertOne({
    _id: "550e8400-e29b-41d4-a716-446655440050",  // тот же UUID, что и
```

```
lecture_id: "550e8400-e29b-41d4-a716-446655440050",
course_name: "Проектирование архитектуры ПО",
lecture_title: "Паттерны проектирования: Singleton, Factory, Observe
lecture_type: "lecture",
order_number: 3,

// Основной текст лекции
content: {
    format: "markdown",
    text: "# Паттерны проектирования\n\n## 1. Singleton\n\nПаттерн S
    word_count: 4520,
    reading_time_minutes: 18
},

// Вложенные материалы разных типов
materials: [
    {
        type: "presentation",
        title: "Слайды к лекции 3",
        file_url: "/files/courses/arch/lecture_03_slides.pdf",
        file_size_mb: 2.3,
        pages: 42
    },
    {
        type: "code_sample",
        title: "Пример Singleton на Python",
        language: "python",
        code: "class Singleton:\n    _instance = None\n    ...",
        runnable: true
    },
    {
        type: "video_link",
        title: "Дополнительное видео по паттернам",
        url: "https://example.com/video/patterns",
        duration_minutes: 45
    },
    {
        type: "exercise",
        title: "Задание для самопроверки",
        questions: [
            {"q": "В чём отличие Factory от Abstract Factory?", "dif
            {"q": "Когда не следует использовать Singleton?", "diffi
        ]
    }
],

// Метаданные
metadata: {
    author: "Иванов И.И.",
    last_updated: ISODate("2026-03-10T14:00:00Z"),
    version: 3,
```

```
        tags: ["паттерны", "GoF", "проектирование", "ООП"],
        difficulty: "intermediate"
    }
});
```

Пояснение: В PostgreSQL материалы хранятся в нормализованном виде (таблица `lecture_materials` с отдельными строками для каждого материала). В MongoDB -- в виде единого документа-агрегата, содержащего все материалы лекции. Это удобно для отображения страницы лекции одним запросом.

5.2. Запросы к MongoDB

```
// 1. Получить все материалы конкретной лекции (один запрос вместо JOIN)
db.lecture_materials.findOne({ lecture_id: "550e8400-e29b-41d4-a716-4466

// 2. Найти все лекции с примерами кода на Python
db.lecture_materials.find({
    "materials.type": "code_sample",
    "materials.language": "python"
}, { lecture_title: 1, "materials.$": 1 });

// 3. Найти лекции с тегом "паттерны"
db.lecture_materials.find({
    "metadata.tags": "паттерны"
}, { lecture_title: 1, course_name: 1 });

// 4. Агрегация: средний объём материалов по курсам
db.lecture_materials.aggregate([
    { $group: {
        _id: "$course_name",
        avg_word_count: { $avg: "$content.word_count" },
        total_lectures: { $sum: 1 },
        total_materials: { $sum: { $size: "$materials" } }
    }},
    { $sort: { total_lectures: -1 } }
]);
```

Шаг 6. Физическая модель -- ElasticSearch (полнотекстовый поиск)

ElasticSearch хранит **тексты лекций и аннотации** для полнотекстового поиска с морфологическим анализом русского языка.

6.1. Создание индекса с маппингом

```
PUT /lectures
{
    "settings": {
        "number_of_shards": 1,
        "number_of_replicas": 0,
        "analysis": {
            "analyzer": {
                "russian_custom": {
                    "type": "custom",
                    "tokenizer": "standard",
                    "filter": ["lowercase", "russian_stop", "russian_ste
                }
            },
            "filter": {
                "russian_stop": {
                    "type": "stop",
                    "stopwords": "_russian_"
                },
                "russian_stemmer": {
                    "type": "stemmer",
                    "language": "russian"
                }
            }
        }
    },
    "mappings": {
        "properties": {
            "lecture_id":      { "type": "keyword" },
            "course_id":       { "type": "keyword" },
            "course_name":     { "type": "text", "analyzer": "russian_cu
                                  "fields": { "keyword": { "type": "keywo
            "title":           { "type": "text", "analyzer": "russian_cu
                                  "fields": { "keyword": { "type": "keywo
            "annotation":      { "type": "text", "analyzer": "russian_cu
            "content_text":    { "type": "text", "analyzer": "russian_cu
            "lecture_type":    { "type": "keyword" },
            "order_number":    { "type": "integer" },
            "tags":            { "type": "keyword" },
            "updated_at":      { "type": "date" }
        }
    }
}
```

Пояснение:

- `russian_custom` анализатор: разбивает текст на слова, приводит к нижнему регистру, убирает стоп-слова («и», «в», «на»), выполняет стемминг (приводит слова к основе: «проектирование» -> «проектирован»)
- Поля `course_name` и `title` имеют мультиполе `keyword` для точной фильтрации и сортировки

6.2. Индексация документа

```
POST /lectures/_doc/550e8400-e29b-41d4-a716-446655440050
{
    "lecture_id": "550e8400-e29b-41d4-a716-446655440050",
    "course_id": "550e8400-e29b-41d4-a716-446655440040",
    "course_name": "Проектирование архитектуры ПО",
    "title": "Паттерны проектирования: Singleton, Factory, Observer",
    "annotation": "Рассматриваются основные порождающие и поведенческие
    "content_text": "Паттерн Singleton гарантирует, что класс имеет толь
    "lecture_type": "lecture",
    "order_number": 3,
    "tags": ["паттерны", "GoF", "проектирование", "ООП"],
    "updated_at": "2026-03-10T14:00:00Z"
}
```

6.3. Поисковые запросы

```
// 1. Полнотекстовый поиск по текстам лекций
// Запрос: "паттерны проектирования singleton"
GET /lectures/_search
{
    "query": {
        "multi_match": {
            "query": "паттерны проектирования singleton",
            "fields": ["title^3", "annotation^2", "content_text"],
            "type": "best_fields",
            "fuzziness": "AUTO"
        }
    },
    "highlight": {
        "fields": {
            "content_text": { "fragment_size": 200, "number_of_fragments
            "annotation": {}
        }
    }
}
```

Пояснение:

- `title^3` -- совпадение в заголовке даёт в 3 раза больший вес, чем в тексте
- `fuzziness: AUTO` -- допускает опечатки (1-2 символа)
- `highlight` -- подсвечивает найденные фрагменты в тексте

```
// 2. Поиск лабораторных, связанных с Docker
GET /lectures/_search
{
    "query": {
        "bool": {
            "must": [
                { "match": { "content_text": "Docker" } }
            ],
            "filter": [
                { "term": { "lecture_type": "lab" } }
            ]
        }
    }
}

// 3. Поиск по аннотациям с фильтром по курсу
GET /lectures/_search
{
    "query": {
        "bool": {
            "must": [
                { "match": { "annotation": "базы данных репликация" } }
            ],
            "filter": [
                { "term": { "course_name.keyword": "Проектирование архит
            ]
        }
    }
}

// 4. Агрегация: количество лекций по типам занятий
GET /lectures/_search
{
    "size": 0,
    "aggs": {
        "by_type": {
            "terms": { "field": "lecture_type" },
            "aggs": {
                "by_course": {
                    "terms": { "field": "course_name.keyword" }
                }
            }
        }
```

```
    }
}
```

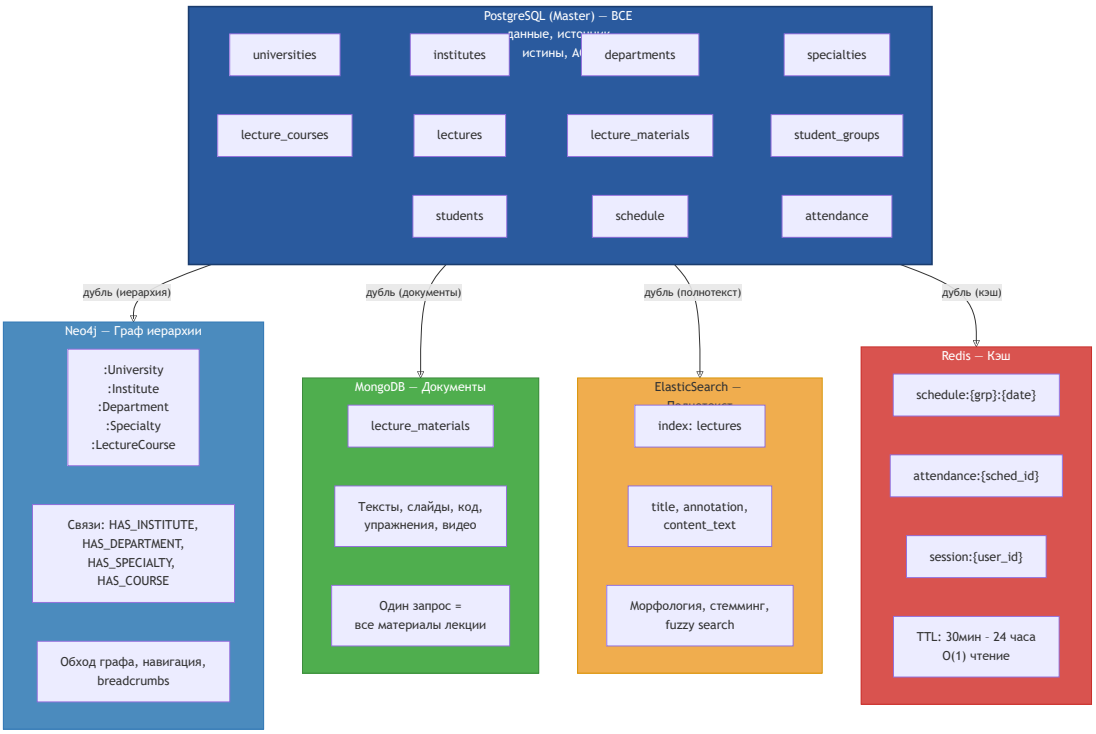
Почему ElasticSearch, а не PostgreSQL для полнотекстового поиска?

PostgreSQL имеет встроенный полнотекстовый поиск (`tsvector` , `tsquery`), но ElasticSearch значительно превосходит его по:

- Качеству ранжирования (BM25 алгоритм)
- Поддержке нечёткого поиска, автодополнения, подсветки
- Скорости поиска по большим объёмам текста
- Возможностям аналитики и агрегации

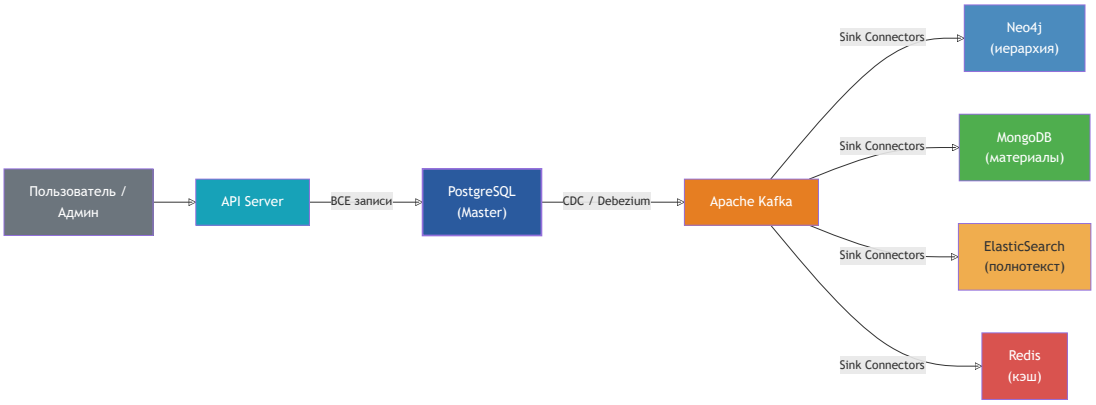
Шаг 7. Сводная карта размещения данных

7.1. Карта данных по хранилищам



7.2. Потоки данных (запись и чтение)

Запись данных (Write path):



> **Примечание:** Синхронизация через CDC реализуется в лабораторных работах 4-6.

Чтение данных (Read path):

Сценарий	Хранилище для чтения	Обоснование
Расписание группы на сегодня	Redis (кэш) -> PostgreSQL (fallback)	Частые чтения, данные меняются редко
Полнотекстовый поиск по лекциям	ElasticSearch	Только ES поддерживает качественный FTS
Навигация по иерархии (breadcrumbs)	Neo4j	Эффективный обход графа
Страница лекции с материалами	MongoDB	Один документ = все материалы
Отметка посещаемости	Redis (буфер) -> PostgreSQL (persist)	Быстрая запись, потом фоновая синхронизация
Отчёт по посещаемости за семестр	PostgreSQL	Сложные агрегации, JOIN, точные данные
Редактирование данных (CRUD)	PostgreSQL	Источник истины, ACID

Шаг 8. Тестовые данные

Для проверки схемы вставим тестовые данные в PostgreSQL:

```
-- Университет
INSERT INTO universities (id, name, short_name, address, website, founde
VALUES ('a0000000-0000-0000-0000-000000000001',
        'МИРЭА — Российский технологический университет',
        'РТУ МИРЭА',
        'г. Москва, проспект Вернадского, 78',
        'https://www.mirea.ru',
        1947);

-- Институт
INSERT INTO institutes (id, university_id, name, short_name, dean)
VALUES ('a0000000-0000-0000-0000-000000000010',
        'a0000000-0000-0000-0000-000000000001',
        'Институт информационных технологий',
        'ИИТ',
        'Сидоров Сергей Сергеевич');

-- Кафедра
INSERT INTO departments (id, institute_id, name, short_name, head, room)
VALUES ('a0000000-0000-0000-0000-000000000020',
        'a0000000-0000-0000-0000-000000000010',
        'Кафедра программной инженерии',
        'КПИ',
        'Иванов Иван Иванович',
        'А-301');

-- Специальность
INSERT INTO specialties (id, name, code, degree_level, duration_years)
VALUES ('a0000000-0000-0000-0000-000000000030',
        'Программная инженерия',
        '09.03.04',
        'bachelor',
        4);

-- Связь кафедра <-> специальность
INSERT INTO department_specialties (department_id, specialty_id, is_prim
VALUES ('a0000000-0000-0000-0000-000000000020',
        'a0000000-0000-0000-0000-000000000030',
        TRUE);  -- КПИ является выпускающей кафедрой для Программной инж

-- Курс лекций
INSERT INTO lecture_courses (id, specialty_id, name, description, semest
VALUES ('a0000000-0000-0000-0000-000000000040',
        'a0000000-0000-0000-0000-000000000030',
        'Проектирование архитектуры ПО',
        'Курс посвящён принципам проектирования архитектуры программного
        6,
        144, 36, 36, 72);
```

```
-- Лекции (занятия)
INSERT INTO lectures (id, course_id, title, annotation, lecture_type, or
('a0000000-0000-0000-0000-000000000051',
 'a0000000-0000-0000-0000-000000000040',
 'Введение в архитектуру ПО. Монолит vs микросервисы',
 'Обзор подходов к архитектуре. Сравнение монолитной и микросервисной ар
 'lecture', 1),

('a0000000-0000-0000-0000-000000000052',
 'a0000000-0000-0000-0000-000000000040',
 'Docker и контейнеризация',
 'Основы Docker, Dockerfile, docker-compose. Контейнеризация приложений.
 'lab', 2),

('a0000000-0000-0000-0000-000000000053',
 'a0000000-0000-0000-0000-000000000040',
 'Паттерны проектирования: Singleton, Factory, Observer',
 'Основные порождающие и поведенческие паттерны из каталога GoF.',
 'lecture', 3),

('a0000000-0000-0000-0000-000000000054',
 'a0000000-0000-0000-0000-000000000040',
 'Развертывание баз данных в Docker',
 'Практика по развертыванию PostgreSQL, Redis, MongoDB, Neo4j, ElasticSe
 'practice', 4);

-- Группа студентов
INSERT INTO student_groups (id, specialty_id, name, enrollment_year, cur
VALUES ('a0000000-0000-0000-0000-000000000060',
        'a0000000-0000-0000-0000-000000000030',
        'ИКБО-01-23',
        2023,
        'Петрова Мария Александровна');

-- Студенты
INSERT INTO students (id, group_id, first_name, last_name, patronymic, e
('a0000000-0000-0000-0000-000000000071', 'a0000000-0000-0000-0000-000000
 'Алексей', 'Козлов', 'Дмитриевич', 'kozlov@mirea.ru', 'СБ-2301001', '20
('a0000000-0000-0000-0000-000000000072', 'a0000000-0000-0000-0000-000000
 'Елена', 'Смирнова', 'Игоревна', 'smirnova@mirea.ru', 'СБ-2301002', '20
('a0000000-0000-0000-0000-000000000073', 'a0000000-0000-0000-0000-000000
 'Дмитрий', 'Волков', 'Андреевич', 'volkov@mirea.ru', 'СБ-2301003', '202

-- Расписание
-- week_start_date вычисляется как понедельник недели: date_trunc('week'
-- 2026-03-15 (воскресенье) -> week_start = 2026-03-09 (понедельник)
-- 2026-03-17 (вторник)     -> week_start = 2026-03-16 (понедельник)
INSERT INTO schedule (id, lecture_id, group_id, scheduled_date, week_sta
('a0000000-0000-0000-0000-000000000081',
 'a0000000-0000-0000-0000-000000000051',
```

```
 'a0000000-0000-0000-0000-000000000060',
 '2026-03-15', '2026-03-09', '08:30', '10:00', 'А-301', 'Иванов И.И.'),
('a0000000-0000-0000-0000-000000000082',
 'a0000000-0000-0000-0000-000000000052',
 'a0000000-0000-0000-0000-000000000060',
 '2026-03-15', '2026-03-09', '10:15', '11:45', 'Б-205', 'Иванов И.И.'),
('a0000000-0000-0000-0000-000000000083',
 'a0000000-0000-0000-0000-000000000054',
 'a0000000-0000-0000-0000-000000000060',
 '2026-03-17', '2026-03-16', '08:30', '10:00', 'Б-205', 'Петров П.П.');

-- Журнал посещений (week_start_date берётся из расписания)
-- Занятия 81 и 82 — неделя 2026-03-09, занятие 83 — неделя 2026-03-16
INSERT INTO attendance (schedule_id, student_id, week_start_date, is_pre
('a0000000-0000-0000-0000-000000000081', 'a0000000-0000-0000-0000-000000
('a0000000-0000-0000-0000-000000000081', 'a0000000-0000-0000-0000-000000
('a0000000-0000-0000-0000-000000000081', 'a0000000-0000-0000-0000-000000
('a0000000-0000-0000-0000-000000000082', 'a0000000-0000-0000-0000-000000
('a0000000-0000-0000-0000-000000000082', 'a0000000-0000-0000-0000-000000
('a0000000-0000-0000-0000-000000000082', 'a0000000-0000-0000-0000-000000
```

Проверка тестовых данных:

```
-- Расписание на 15 марта
SELECT * FROM v_full_schedule WHERE scheduled_date = '2026-03-15';

-- Посещаемость
SELECT * FROM v_student_attendance_stats;
```

Важное замечание

> **Представленные в данной работе модели данных являются примерными и демонстрационными.** Они иллюстрируют принципы проектирования логической и физической структуры данных с распределением по нескольким хранилищам.
>
> При выполнении лабораторных работ 1-3 студент должен **переработать и адаптировать** эти модели с учётом конкретных требований каждой лабораторной работы:
>
> - **Лабораторная работа 1 (Микросервисы, API Gateway):** декомпозировать схему данных на отдельные микросервисы (например,

> сервис расписания, сервис студентов, сервис посещений), каждый со своей БД. Определить, какие таблицы принадлежат какому сервису. Спроектировать REST API эндпоинты (минимум 10) для работы с данными. Реализовать JWT-аутентификацию и маршрутизацию через API Gateway.
>
> - **Лабораторная работа 2 (Очередь сообщений):** определить, какие события между сервисами требуют асинхронной обработки (например, при создании записи в расписании -- уведомление сервиса студентов; при отметке посещения -- обновление статистики). Добавить RabbitMQ для межсервисного взаимодействия.
> - **Лабораторная работа 3 (Шардирование и репликация):** оптимизировать модель данных для горизонтального масштабирования. Определить ключи шардирования (например, `group_id` для таблицы attendance, `department_id` для организационной структуры). Настроить репликацию для отказоустойчивости. Учесть, что партицирование attendance по неделям уже заложено в модели -- при шардировании это можно использовать как основу.
>
> Схема данных должна быть скорректирована под задачи конкретной лабораторной работы: добавлены недостающие поля (например, `created_by`, `version` для отслеживания изменений), удалены избыточные, оптимизированы индексы под реальные паттерны запросов.

Требования к отчёту

Отчёт по практической работе должен содержать:

1. **ER-диаграмма** (логическая модель) -- все сущности, атрибуты, связи с кардинальностями
2. **Физическая схема PostgreSQL** -- DDL-скрипты создания всех таблиц с комментариями
3. **Схема размещения данных по хранилищам** -- карта, показывающая какие данные где хранятся и почему
4. **Структура данных в Redis** -- ключи, типы данных, TTL, примеры команд
5. **Модель графа Neo4j** -- узлы, связи, свойства, примеры Cypher-запросов
6. **Структура коллекции MongoDB** -- пример документа с пояснениями
7. **Маппинг ElasticSearch** -- настройки индекса, анализаторы, примеры поисковых запросов

8. **Обоснование выбора** -- для каждого участка данных объяснение, почему выбрано именно это хранилище
9. **Тестовые данные** -- заполненная БД с примерами запросов к каждому хранилищу
10. **Диаграмма потоков данных** -- как данные записываются и читаются из разных хранилищ

Контрольные вопросы

1. Что такое polyglot persistence? В чём преимущество использования нескольких СУБД вместо одной?
2. Почему PostgreSQL выбран как основное (master) хранилище? Какие свойства PostgreSQL делают его подходящим для этой роли?
3. Зачем дублировать данные из PostgreSQL в Redis, если PostgreSQL и так содержит все данные?
4. Какие данные из схемы хранятся **только** в ElasticSearch и не дублируются в PostgreSQL? Почему?
5. Объясните, почему иерархия «Университет -> Институт -> Кафедра -> Специальность -> Курс» размещена в Neo4j. Какие запросы к графу эффективнее, чем JOIN в SQL?
6. Почему материалы лекций хранятся в MongoDB как документы, а не в PostgreSQL как строки таблицы?
7. Что такое TTL в Redis? Зачем расписание кэшируется с TTL = 1 час?
8. Как обеспечивается консистентность данных между PostgreSQL и NoSQL-хранилищами?
9. Что произойдёт, если Redis станет недоступен? Как система продолжит работу?
10. Объясните разницу между логической (ER) и физической моделью данных.
11. Что такое CDC (Change Data Capture) и как он связан с синхронизацией данных между хранилищами?
12. Какой паттерн кэширования используется для расписания? Опишите его работу.
13. Почему для поиска по текстам лекций используется ElasticSearch, а не `tsvector` в PostgreSQL?
14. Как UUID помогает при работе с несколькими хранилищами?